Create a project in VS Code

1. Open VS Code.
2. Select File → Open Folder.
3. Create and open a folder named `gemini_model_lab`.
4. Select Terminal → New Terminal.
5. Check your Python installation.

Run the following commands in the VS Code terminal:

```
python --version
```

If that does not work on Windows, try:

```
py --version
```

You should see a Python version number.

3. Create a virtual environment

A virtual environment keeps this project's Python packages separate from packages used by other projects.

Run in the VS Code terminal:

```
python -m venv .venv
```

If you use the `py` launcher instead:

```
py -m venv .venv
```

Activate the environment in PowerShell:

```
.\.venv\Scripts\Activate.ps1
```

If activation succeeds, your terminal prompt should show `(.venv)`.

If PowerShell blocks activation

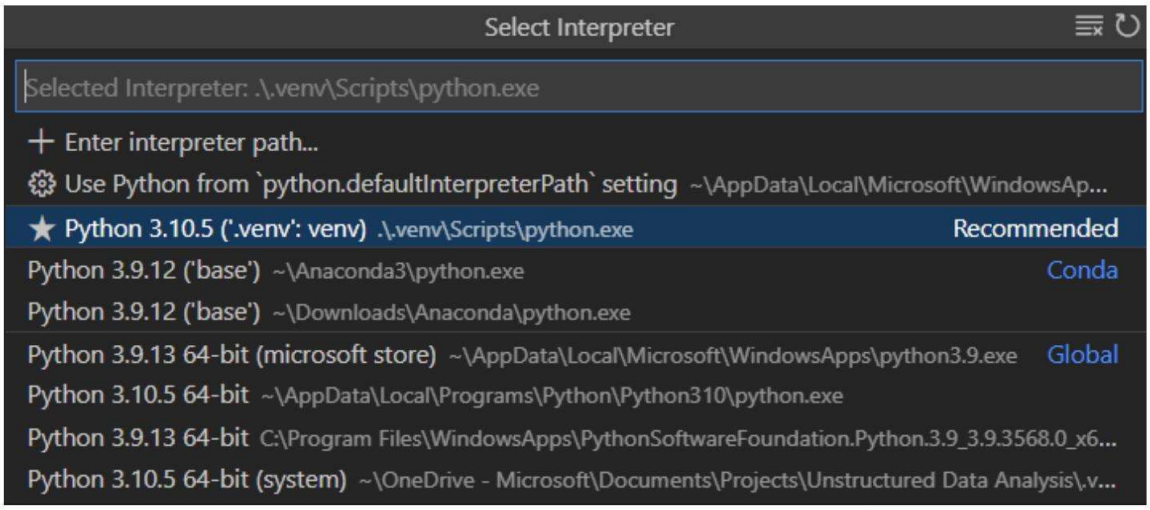
You can use Command Prompt in VS Code instead and run:

```
.venv\Scripts\activate.bat
```

Alternatively, continue without activating by using `.venv\Scripts\python.exe` to run Python commands.

4. Select the Python interpreter

2. Ensure your new environment is selected by using the **Python: Select Interpreter** command from the **Command Palette**.



stackoverflow.com

1. Install the Python extension published by Microsoft in VS Code.
2. Press `Ctrl + Shift + P`.
3. Search for `Python: Select Interpreter`.
4. Select the interpreter inside your project folder: `.venv`.
5. Open a new terminal if necessary.

This ensures that packages are installed into and used from the same environment.

5. Install Google Gen AI SDK and dotenv

Run:

```
python -m pip install --upgrade pip
python -m pip install google-genai python-dotenv
```

Verify the installations:

```
python -m pip show google-genai
python -m pip show python-dotenv
```

6. Create a `.env` file to store the API key securely

In the VS Code Explorer, create a new file named:

```
.env
```

Add the following line, replacing the placeholder with your actual key:

```
GEMINI_API_KEY=your_actual_api_key_here
```

Do not add spaces around the = sign, and do not share this key with students or upload it to GitHub.

Protect your API key

Create another file named `.gitignore`:

```
.env
.venv/
__pycache__/
*.pyc
```

This helps prevent your API key and local environment from being committed to Git.

Important: `.gitignore` does not encrypt the key. Keep `.env` private, and revoke the key if it is accidentally exposed.

7. Create your first Gemini Python program

Create a file named `test_gemini.py`.

Check different Models :

https://ai.google.dev/gemini-api/docs/models

Compare different Gemini models using the same prompt

he experiment should hold the prompt constant while changing the model. Measure:

- Latency: time taken for the API request to complete.
- Quality: how correct, clear, complete, and useful the response is.
- Token usage: input and output token counts, when returned by the API.
- Cost: estimated API cost based on the applicable pricing tier and token rates.

Use these three model IDs as initial candidates, subject to availability:

Model	What to investigate
`gemini-2.5-flash-lite`	Efficiency for simple tasks
`gemini-2.5-flash`	Balance of speed and quality
`gemini-2.5-pro`	Quality on complex tasks

Model names and availability can change. Confirm the currently supported models before running the comparison.

Create `compare_models.py`

Prices models gemin: https://ai.google.dev/gemini-api/docs/pricing

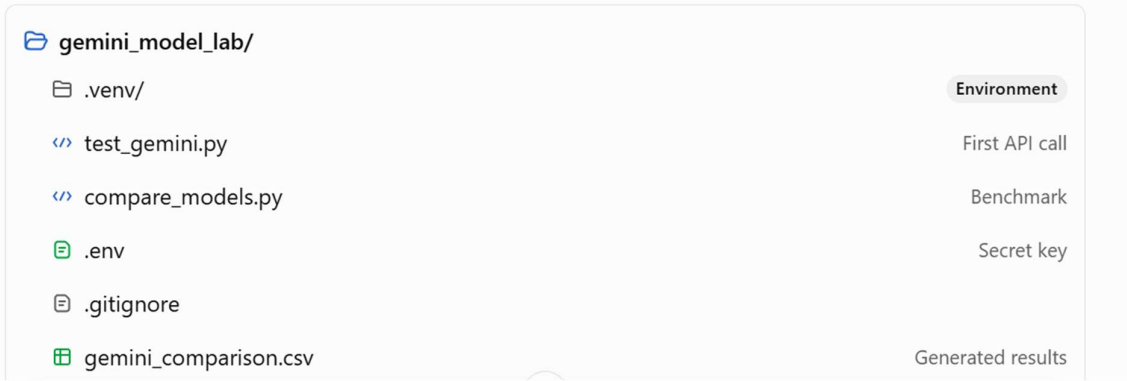
The basic calculation is:

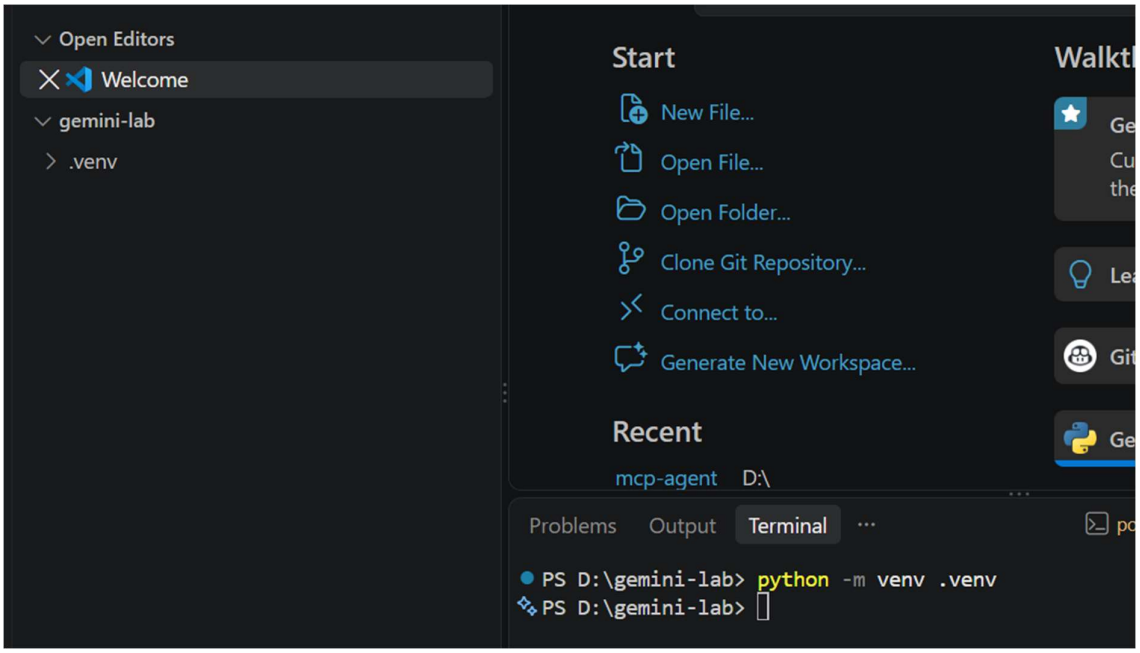
$$\text{Cost} = \frac{\text{Input tokens} \times \text{Input rate}}{1{,}000{,}000} + \frac{\text{Output tokens} \times \text{Output rate}}{1{,}000{,}000}$$

For example, suppose a hypothetical model charges $0.50 per million input tokens and $2.00 per million output tokens. If one request uses 1,000 input tokens and 500 output tokens:

- Input cost = $0.0005
- Output cost = $0.0010
- Total estimated cost = **$0.0015**

These rates are illustrative, not actual current Gemini prices. The actual charge depends on the selected model, pricing tier, token categories, and any additional tools used. If you are on a free tier, do not assume that your measured usage translates directly into a bill.





PS D:\gemini-lab> python -m venv .venv

PS D:\gemini-lab> .\.venv\Scripts\Activate.ps1

- Install the Python extension published by Microsoft in VS Code.
- Press `Ctrl + Shift + P`.
- Search for `Python: Select Interpreter`.
- Select the interpreter inside your project folder: `.venv`.
- Open a new terminal if necessary.

. Parameters we will experiment with

Parameter	What you will learn
Model	How different Gemini models respond to the same prompt
Temperature	How response variability changes
System instruction	How to control the model's role and behavior
Context	How additional information influences answers
Input prompt	How different questions affect responses
Max output tokens	How to limit response length
Top-P	How sampling behavior can change
Latency and token usage	How to measure API performance